MeteoCity

Monitoring City Weather — Milan & Rome

POLITECNICO DI MILANO

Software Engineering for Geoinformatics

Software Release Document

MeteoCity

City-Level Weather Monitoring using Meteostat

Academic Year 2025/26

Group 9

Authors:

Ayman Abdelgadir

Mobina Faraji

Hangyun Wang

June 2026

Table of Contents

1. Introduction **4**
1.1 Purpose 4
1.2 Product Overview 4
1.3 Release Scope 4
1.4 Intended Audience 4
1.5 Acronyms and Definitions 5
2. Released Software Overview **6**
2.1 Release Identification 6
2.2 Repository Structure 6
2.3 Released Components 8
3. System Requirements **9**
3.1 Required Software 9
3.2 Python Dependencies 9
3.3 External Services and Network Access 10
3.4 Local Ports and URLs 10
4. Installation and Configuration **11**
4.1 Clone the Repository 11
4.2 Set Up Anaconda Python Environment 11
4.3 Install Dependencies 11
4.4 Configure the .env File 11
4.5 Prepare the PostgreSQL Database 12
4.6 Backend Auto-Creates Schema 12
5. Data Ingestion **13**
5.1 Ingestion Purpose 13
5.2 Mock Mode (Default — No API Key) 13
5.3 Live Meteostat Mode 13
5.4 30-Day Constraint and Synthetic Supplement 13
6. Running the Released Software **14**
6.1 Recommended Startup Sequence 14
6.2 Start the Flask Backend 14
6.3 Start the Dashboard 14
6.4 Stopping the Services 14

7. REST API Reference 15
8. Dashboard User Guide 16
8.1 Login and Registration 16
8.2 Typical Workflow 16
8.3 Role-Based Access 16
9. Testing and Release Validation 17
9.1 Running the Test Suite 17
9.2 Manual Acceptance Checklist 17
10. Known Limitations and Troubleshooting 19
10.1 Known Limitations 19
10.2 Troubleshooting Guide 20
11. Maintenance and Future Work 21
12. Generative AI Usage Declaration 21
13. References 22
Appendix A — Quick Start Summary 23
Appendix B — Environment File Template 23

List of Tables

Table 1: Acronyms and Definitions 5
Table 2: Release Identification 6
Table 3: Repository Structure 7
Table 4: Released Components 8
Table 5: Required Software 9
Table 6: Python Dependencies 9
Table 7: Local Ports and URLs 10
Table 8: REST API Endpoints 15
Table 9: Role-Based Access Summary 16
Table 10: Unit Test Cases 17
Table 11: Troubleshooting Guide 20

1. Introduction

1. Purpose

This Software Release Document (SRD) describes how to install, configure, run, test, and maintain the released MeteoCity software. It accompanies the final software release submitted through the group GitHub repository for the MeteoCity project.

The SRD focuses on operational release information. It does not replace the Requirement Analysis and Specification Document (RASD) or the Design Document (DD); it explains how the implemented software can be executed and evaluated by course instructors and teaching assistants.

2. Product Overview

MeteoCity is a client-server application for city-level weather monitoring. The system retrieves, stores, and exposes historical station-based meteorological observations for Milan and Rome, and provides interactive geographic maps and temporal analysis through a role-based web-based dashboard.

The release follows the three-tier architecture defined in the DD: a PostgreSQL data tier for meteorological storage, a Flask REST API application tier handling authentication and business logic, and an interactive web-based dashboard presentation tier supporting spatial mapping, temporal filters, and analytical charts.

3. Release Scope

- Selected monitored cities: Milan and Rome.
- Station metadata: station identifiers, names, geographic coordinates, elevation, and city association.
- Hourly meteorological observations: temperature (°C), humidity (%), wind speed (km/h), and weather condition code.
- PostgreSQL database via SQLAlchemy ORM; backend auto-creates schema on first run.
- Flask backend with JWT-based authentication and REST API endpoints.
- Interactive web-based dashboard for map visualisation, filtering, trend analysis, statistics, and CSV export.
- Data ingestion module supporting both RapidAPI/Meteostat live mode and synthetic mock mode.
- Test suite (pytest) for backend API and database validation.

1.4 Intended Audience

- Course instructors and teaching assistants evaluating the final project release.
- Group members demonstrating or maintaining the project.
- Future developers who need to understand the release structure and reproduce the local environment.

1.5 Acronyms and Definitions

Table 1: Acronyms and Definitions

Term	Definition
API	Application Programming Interface.
DD	Design Document.
FR	Functional Requirement.
JWT	JSON Web Token — used for secure session authentication.
NFR	Non-Functional Requirement.
ORM	Object-Relational Mapper (SQLAlchemy).
RASD	Requirement Analysis and Specification Document.
REST	Representational State Transfer.
SRD	Software Release Document.
TR	Technical Requirement.

2. Released Software Overview

1. Release Identification

Table 2: Release Identification

Item	Release Information
Product Name	MeteoCity
Release Version	1.0 — Final Academic Release
Repository URL	https://github.com/Ayman1017/Software-Engineering-Project-Group_9
Primary Branch	master
Release Type	Academic local/development release
Target Deployment	Local machine with PostgreSQL, Flask backend, and web-based dashboard

2.2 Repository Structure

The following table describes the top-level directory and file layout of the Software-Engineering-Project-Group_9 repository as submitted for final evaluation. The structure reflects the actual committed files in the master branch.

```
Software-Engineering-Project-Group_9/
|-- backend/
| |-- __init__.py
| |-- app.py # Flask entry point; auto-creates DB schema
| |-- config.py # Environment configuration mapper
| |-- models.py # SQLAlchemy ORM table definitions
| |-- clean_db.py # Utility to reset the database
| |-- .env # Local secrets (gitignored -- NOT committed)
| |-- .env.example # Committed template with placeholder values
| |-- ingestion/
| | `-- fetch_hourly.py # Meteostat hourly observations fetcher
| `-- routes/
| |-- auth.py # Registration and JWT login
| |-- cities.py # City resource endpoints
| |-- stations.py # Weather station discovery
| |-- observations.py # Observation record fetchers
| |-- statistics.py # Analytical aggregate routes
| |-- map_data.py # Spatial data for map display
| `-- export.py # CSV export generator
|-- dashboard/
| |-- meteocity_dashboard.py # Interactive web-based dashboard
| `-- bg.png # Dashboard background asset
|-- tests/
| `-- test_auth.py # Authentication regression tests
|-- .gitignore
|-- FINAL_VERSION_DD.pdf
|-- FINAL_VERSION_RASD.pdf
|-- project_description_Meteostat.pdf
|-- requirements.txt
`-- README.md
```

Table 3: Repository Structure

Path / File	Description
backend/app.py	Flask application entry point. Auto-creates all database tables on first run via SQLAlchemy db.create_all().
backend/models.py	ORM model definitions for Users, Cities, WeatherStations, Observations, and DataVariables.
backend/clean_db.py	Utility script to reset and clean the database during development.
backend/.env	Local environment file containing real credentials. Excluded from version control via .gitignore. Must be created manually on each machine.
backend/.env.example	Committed placeholder template. Copy to backend/.env and fill in real values before running.
backend/ingestion/	Data ingestion helpers for fetching Meteostat hourly observations.
backend/routes/	Flask Blueprint modules: auth, cities, stations, observations, statistics, map_data, export.
dashboard/meteocity_dashboard.py	Interactive web-based dashboard (presentation tier).

Path / File	Description
tests/test_auth.py	Pytest-based authentication regression tests.
requirements.txt	All Python package dependencies.
README.md	Repository overview and startup instructions.

2.3 Released Components

Table 4: Released Components

Component	Implementation	Main Responsibility
Database Tier	SQLAlchemy models in backend/models.py; PostgreSQL database meteocity.	Stores user accounts, city records, station metadata, hourly observations, and data variable definitions.
Backend Layer	Flask application in backend/app.py with route modules under backend/routes/.	Exposes authenticated REST endpoints returning JSON; enforces role-based access control.
Data Ingestion	backend/ingestion/fetch_hourly.py, triggered via the dashboard's Show Analysis action.	Fetches real observations from Meteostat/RapidAPI; falls back to synthetic data when no key is configured.
Dashboard	dashboard/meteocity_dashboard.py	Interactive web-based interface: maps, filters, charts, statistics, CSV export, role-based views.
Test Suite	tests/test_auth.py; pytest framework.	Validates registration, login, JWT issuance, and role-based endpoint access.

3. System Requirements

1. Required Software

Table 5: Required Software

Software	Version / Notes
Operating System	Windows 10/11 (primary development environment). macOS and Linux also supported.
Python (Anaconda)	Python 3.10 or later via the Anaconda distribution. Tested and recommended for Windows.
PostgreSQL	Version 14 or higher installed locally. Database must be named meteocity (not meteocity_db).
Git	Required to clone the repository from GitHub.
Web Browser	Google Chrome, Firefox, or Safari for dashboard access and map tile rendering.
RapidAPI Account	Required only for live Meteostat ingestion. Free tier: 500 requests/month. Mock mode works without a key.

3.2 Python Dependencies

All packages are listed in requirements.txt at the repository root. Install them using the command in Section 4.3.

Table 6: Python Dependencies

Package	Version	Purpose
Flask	3.0.3	Lightweight WSGI framework; REST API backend.
Flask-JWT-Extended	4.6.0	JWT token management with role claims.
Flask-SQLAlchemy	3.1.1	SQLAlchemy integration for Flask.
SQLAlchemy	≥ 2.0.30	ORM; auto-creates schema; ON CONFLICT upsert support.
psycopg2-binary	≥ 2.9.9	PostgreSQL adapter for Python.
python-dotenv	1.0.1	Loads environment variables from backend/.env.
bcrypt	4.1.3	Secure password hashing.
requests	2.32.3	HTTP client for Meteostat RapidAPI calls.
pandas	≥ 2.2.2	Data manipulation and preprocessing.
folium	0.16.0	Leaflet.js-backed interactive maps.
plotly	5.22.0	Interactive time-series and bar charts.
Flask-Cors	4.0.1	Cross-origin resource sharing.
streamlit	≥ 1.35.0	Web-based dashboard framework.
streamlit-folium	≥ 0.21.0	Folium map integration for the dashboard.

3.3 External Services and Network Access

- Meteostat via RapidAPI (meteostat.p.rapidapi.com): used for live hourly observation ingestion. Requires outbound internet on port 443.
- Map tile providers: internet connection required by the dashboard to resolve OpenStreetMap base tiles via Folium/Leaflet.
- Local loopback: the dashboard communicates with the Flask backend over localhost. Ports 5000 and 8501 must be free.
- Mock ingestion mode populates the database with synthetic data and requires no external network access.

3.4 Local Ports and URLs

Table 7: Local Ports and URLs

Service	Default URL / Port	Purpose
Flask Backend	http://localhost:5000	REST API and authentication server.
Dashboard	http://localhost:8501	Interactive web-based dashboard.
PostgreSQL	localhost:5432	Database server.

4. Installation and Configuration

Follow these steps sequentially to configure the development stack locally. All commands are shown for Windows (Anaconda). macOS/Linux equivalents are noted where they differ.

1. Clone the Repository

```
git clone https://github.com/Ayman1017/Software-Engineering-Project-Group_9.git
cd Software-Engineering-Project-Group_9
```

4.2 Set Up Anaconda Python Environment

The project is developed and tested using Anaconda Python on Windows. Use the base Anaconda environment or create a dedicated conda environment.

Option A — Use the base Anaconda environment (simplest):

```
# Open Anaconda Prompt and navigate to the project
cd Software-Engineering-Project-Group_9
```

Option B — Create a dedicated conda environment:

```
conda create -n meteocity python=3.11
conda activate meteocity
```

macOS/Linux alternative (standard venv):

```
python3 -m venv venv
source venv/bin/activate
```

4.3 Install Dependencies

```
pip install --upgrade pip
pip install -r requirements.txt
```

All packages listed in Section 3.2 will be installed. Ensure requirements.txt has one package per line if pip raises parsing errors.

4.4 Configure the .env File

The backend reads all configuration from backend/.env. This file contains database credentials, the JWT secret key, and the RapidAPI key. It is excluded from version control via .gitignore and must be created manually on every new machine.

A committed template backend/.env.example is provided with placeholder values and inline comments. Copy it to backend/.env and fill in your real values:

```
# Windows
copy backend\.env.example backend\.env

# macOS / Linux
cp backend/.env.example backend/.env
```

Then edit backend/.env and set:

```
DATABASE_URL=postgresql://postgres:<your_password>@localhost:5432/meteocity
JWT_SECRET_KEY=<change-this-to-a-long-random-secret>
RAPIDAPI_KEY=<your_rapidapi_key_or_leave_empty_for_mock_mode>
RAPIDAPI_HOST=meteostat.p.rapidapi.com
FLASK_ENV=development
```

The database name must be meteocity (not meteocity_db). If RAPIDAPI_KEY is left empty or set to the placeholder, the system automatically falls back to synthetic mock data.

4.5 Prepare the PostgreSQL Database

Create the database named meteocity before starting the backend. No PostGIS extension is required.

Using psql:

```
psql -U postgres
CREATE DATABASE meteocity;
\q
```

Using pgAdmin:

- Open pgAdmin and connect to your local PostgreSQL server.
- Right-click Databases → Create → Database.
- Enter meteocity as the database name and click Save.

4.6 Backend Auto-Creates Schema

The backend automatically creates all required database tables on first run via SQLAlchemy db.create_all(). No manual migration commands are needed. The meteocity database instance must exist before the backend is started.

5. Data Ingestion

1. Ingestion Purpose

The ingestion module populates the database with city records, weather station metadata, data variable definitions, and hourly observation records. Ingestion is triggered automatically when the dashboard calls the /api/observations endpoint after the user clicks Show Analysis.

Four hardcoded Meteostat weather stations are used: Milano Malpensa (16080), Milano Linate (16066), Roma Fiumicino (16242), and Roma Ciampino (16239).

2. Mock Mode (Default — No API Key)

When RAPIDAPI_KEY is absent, empty, or set to a placeholder value in backend/.env, the system automatically uses mock mode. Synthetic but realistic hourly weather observations are generated for all four stations and stored in the database. This mode is suitable for demonstrations and development testing without consuming RapidAPI quota.

Mock data uses a fixed set of weather condition codes in {1, 2, 3, 4, 5, 21, 61}.

3. Live Meteostat Mode

When a valid RAPIDAPI_KEY is configured in backend/.env, the system requests real hourly observations from the Meteostat API via RapidAPI. Upsert logic (ON CONFLICT DO NOTHING) ensures real data is never overwritten by subsequent synthetic fallback operations.

The free Meteostat plan via RapidAPI provides 500 requests per month. Manage requests carefully to avoid quota exhaustion.

4. 30-Day Constraint and Synthetic Data Supplement

The Meteostat API enforces a hard 30-day maximum window per request (TR8 in the RASD). For date ranges exceeding 30 days, the system automatically supplements real observations with synthetic values to fill the missing period. Real Meteostat data is only guaranteed for ranges of 30 days or fewer. This is an acknowledged external constraint documented in both the RASD and DD, not a bug.

6. Running the Released Software

1. Recommended Startup Sequence

Launch services in this order to prevent connection errors:

1. Ensure PostgreSQL is running and the meteocity database exists.
2. Ensure backend/.env is correctly configured (Section 4.4).
3. Open Terminal 1 and start the Flask backend (Section 6.2).
4. Open Terminal 2 and start the dashboard (Section 6.3).
5. Open http://localhost:8501 in a web browser.

6.2 Start the Flask Backend

In Terminal 1, from the project root directory:

```
cd backend
python app.py
```

Expected: the backend listens on http://localhost:5000 and logs confirmed table creation on first run. Leave this terminal running.

6.3 Start the Dashboard

In Terminal 2, from the project root directory (not backend/):

```
streamlit run dashboard\meteocity_dashboard.py
```

The dashboard opens automatically at http://localhost:8501. The Flask backend must already be running before starting the dashboard.

6.4 Stopping the Services

Press Ctrl+C in Terminal 1 to stop the Flask backend. Press Ctrl+C in Terminal 2 to stop the dashboard. If using a conda environment, run conda deactivate when finished.

7. REST API Reference

All protected API endpoints require a JWT token in the Authorization header after login:

```
Authorization: Bearer <access_token>
```

Table 8: REST API Endpoints

Method	Endpoint	Auth	RASD	Description
GET	/health	No	—	Health check; returns {"status":"ok"}.
POST	/auth/register	No	FR(Users)	Register a new user (username, password, role).
POST	/auth/login	No	FR(Users)	Authenticate user; returns JWT access token.
GET	/auth/logout	Yes	FR(Users)	Logs out the current user.
GET	/auth/profile	Yes	FR(Users)	Returns current user profile and role.
GET	/api/cities	Yes	FR3	Returns list of monitored cities (Milan, Rome).
GET	/api/stations	Yes	FR2, TR5	Returns station metadata; optional city_name filter.
GET	/api/observations	Yes	FR1–FR3	Returns hourly observations with date/station filters.
GET	/api/trend_data	Yes	FR5, FR7	Returns aggregated time-series for trend visualisation.
GET	/api/statistics	Gov	FR5	Statistical summary (mean, max, min, std). Government only.
GET	/api/map_data	Yes	FR4	Station data for map display with latest observation.
GET	/api/export_csv	Yes	TR7	Exports filtered observations as downloadable CSV.
GET	/api/variables	Yes	FR6	Returns list of available meteorological variables.

Example API Usage

Register a new user:

```
curl -X POST http://localhost:5000/auth/register ^
-H "Content-Type: application/json" ^
-d  "{\"username\":\"testuser\",\"password\":\"test123\",\"role\":\"government\"}"
```

Login and retrieve JWT token:

```
curl -X POST http://localhost:5000/auth/login ^
-H "Content-Type: application/json" ^
-d "{\"username\":\"testuser\",\"password\":\"test123\"}"
```

Query stations for Milan:

```
curl -H "Authorization: Bearer <access_token>" ^
"http://localhost:5000/api/stations?city_name=Milan"
```

8. Dashboard User Guide

1. Login and Registration

The dashboard displays a login and registration interface when first launched. New users select a role (Student/Researcher, NGO, or Government Official) and register with a username and password (minimum 6 characters). After logging in, a JWT session token is stored for the duration of the session.

The login screen features MeteoCity branding over a Milan and Rome cityscape background. A role description panel informs new users of available access levels.

2. Typical Workflow

1. Register or log in via the login/registration screen.
2. Select a city (Milan or Rome) and date range using the sidebar filtering widgets.
3. Select the weather variable (Temperature, Humidity, or Wind Speed).
4. Click Show Analysis; the dashboard queries the REST API and renders colour-coded station markers on the map.
5. Navigate to the Trend Analysis tab for hourly readings with a 24-hour moving average and weekly average bar charts.
6. Navigate to the Statistics tab (Government Official and NGO roles) for descriptive statistics per station.
7. Use the Export CSV tab to download the currently filtered observations.
8. Click Logout to end the session.

8.3 Role-Based Access

Table 9: Role-Based Access Summary

Feature	Student/Researcher	NGO	Government Official
Map & Stations	✓	✓	✓
Query & Filtering	✓	✓	✓
Trend Analysis	✗	✓	✓
Statistics Dashboard	✗	✗	✓
Export CSV	✓	✓	✓
User Registration	✓	✓	✓
All API Endpoints	✗	✗	✓

9. Testing and Release Validation

1. Running the Test Suite

A comprehensive test suite is implemented using the pytest framework. All test files are in the tests/ directory. The Flask backend must be running before executing tests that make HTTP requests.

```
pytest tests/ -v
```

Table 10: Unit Test Cases

Test ID	Component	Description	Expected Outcome
UT01	Backend API	POST /auth/register with valid credentials	201 Created; user saved in database.
UT02	Backend API	POST /auth/login with correct credentials	200 OK; JWT token returned.
UT03	Backend API	POST /auth/login with incorrect password	401 Unauthorized.
UT04	Backend API	GET /api/stations returns station list	200 OK; JSON array of stations.
UT05	Backend API	GET /api/observations with valid filters	200 OK; filtered records returned.
UT06	Backend API	GET /api/statistics without government role	403 Forbidden.
UT07	Backend API	GET /api/export_csv with valid filters	200 OK; CSV file returned.
UT08	Database	Database connection and table creation	All tables created without error.
UT09	Database	Insert and retrieve WeatherStation record	Record persists and is returned correctly.
UT10	Database	Insert and retrieve Observation with timestamp filter	Correct records returned for date range.
UT11	Database	Duplicate observation insert (upsert logic)	Duplicate skipped; no error raised.
UT12	Meteostat	Fetch hourly observations for valid station and range	Observations returned and stored in database.

9.2 Manual Acceptance Checklist

Verify the following before final submission and demonstration:

- PostgreSQL is running and the meteocity database exists.

- backend/.env is correctly configured with DATABASE_URL, JWT_SECRET_KEY, and (optionally) RAPIDAPI_KEY.
- Flask backend starts without errors on http://localhost:5000.
- GET /health returns {"status": "ok"}.
- A user can register with each role (student, ngo, government) and log in successfully.
- Protected endpoints reject requests without a valid JWT token (401 Unauthorized).
- The dashboard connects to localhost:5000 and loads city/station data.
- Map markers are visible and colour-coded for Milan and Rome.
- Observation charts render correctly for selected date ranges.
- Statistics tab is accessible for Government Official and NGO but hidden for Student.
- CSV export downloads a file with the correct columns for the selected variable.
- The GitHub repository contains RASD, DD, SRD, and all source code.

10. Known Limitations and Troubleshooting

1. Known Limitations

- Academic local release: The system is a local/development academic release and is not configured for production deployment or multi-user concurrent access.
- Meteostat 30-day API constraint: The Meteostat API enforces a hard 30-day maximum window per request (TR8 in the RASD). For ranges exceeding 30 days, real observations are supplemented with synthetic data automatically.
- RapidAPI quota: The free Meteostat plan allows 500 requests per month. Repeated live ingestion may exhaust this quota.
- JWT logout blocklist: The JWT blocklist is stored in memory. Revoked tokens are not persisted after backend restart.
- Hardcoded stations and cities: The system uses four hardcoded Meteostat station IDs. Adding new cities or stations requires code changes.
- No real-time data: The system does not provide real-time forecasting, live streaming, predictive modelling, or alerts.
- NGO registration case sensitivity: The frontend sends role value 'NGO' while the backend validates against 'ngo'. If NGO registration fails via the form, register directly via the API with role='ngo' (lowercase).

10.2 Troubleshooting Guide

Table 11: Troubleshooting Guide

Problem	Likely Cause	Recommended Solution
Backend cannot start	DATABASE_URL wrong, PostgreSQL stopped, or database meteocity missing.	Start PostgreSQL, create the meteocity database, and verify DATABASE_URL in backend/.env.
'meteocity_db' not found	Database name mismatch.	Ensure DATABASE_URL uses the name meteocity, not meteocity_db.
.env not found / config error	backend/.env does not exist on this machine.	Copy backend/.env.example to backend/.env and fill in real values (Section 4.4).
Dashboard cannot connect to Flask	Backend not running on localhost:5000.	Start cd backend && python app.py before launching the dashboard.
No observations / empty charts	Ingestion not triggered or date range has no records.	Click Show Analysis; select a date range within the available data window.
Statistics returns 403	User does not have the government role.	Register or log in with role='government'.
RapidAPI calls fail	Invalid key, quota reached, or no internet.	Verify RAPIDAPI_KEY and RAPIDAPI_HOST in backend/.env; leave empty to use mock mode.
NGO registration fails	Frontend sends 'NGO'; backend expects 'ngo' (case mismatch).	Register via API with role='ngo' (lowercase) using curl or Postman.
Map tiles do not load	No internet connection.	Ensure internet access is available for OpenStreetMap tiles to render.

11. Maintenance and Future Work

Maintenance Guidelines

- Re-run pytest tests/ -v after any modification to routes, models, or ingestion logic.
- Keep backend/.env private and never commit real secrets to GitHub. Use backend/.env.example for documentation.
- Monitor RapidAPI usage when running live ingestion to avoid exceeding the 500 requests/month free quota.
- Keep RASD, DD, SRD, README, and implementation consistent before any final submission.
- Avoid hardcoding passwords, API keys, or machine-specific paths in committed files.

Future Improvements

- Resolve the NGO registration case-sensitivity issue in the frontend form.
- Implement persistent JWT token revocation (e.g., Redis) to survive backend restarts.
- Add Docker and Docker Compose to bundle PostgreSQL, Flask, and the dashboard for one-command startup.
- Expand the pytest suite with integration tests covering all REST API endpoints.
- Add GitHub Actions for automatic linting and test execution on every push.
- Improve date range handling to blend real and synthetic data more seamlessly for longer periods.

12. Generative AI Usage Declaration

In compliance with the Politecnico di Milano Code of Academic Conduct and the guidelines issued by the MeteoCity project for Academic Year 2025/26, Group 9 declares the following use of Generative AI tools during the development of the MeteoCity project.

Tools Used

- Claude (Anthropic) — used as an AI programming assistant and documentation aid.
- GitHub Copilot — used for code auto-completion suggestions within the VS Code IDE.

Scope of Use

- Code assistance: AI tools suggested boilerplate code, helped debug syntax errors, and proposed function implementations. All generated code was reviewed, understood, and modified by team members.
- Documentation: AI tools assisted in drafting and refining sections of the RASD, DD, and SRD. All factual content reflects the actual system implemented by the team.
- No AI-generated content was accepted uncritically or without verification.

Responsibility Statement

All team members take full responsibility for the content of this document and the accompanying software artefacts. The use of Generative AI tools was limited to productivity enhancement and did not substitute for the intellectual contribution, design decisions, or engineering judgement of the team.

13. References

1 Meteostat API via RapidAPI. https://rapidapi.com/meteostat/api/meteostat/

2 Flask Documentation. Pallets Projects, 2024. https://flask.palletsprojects.com/

3 SQLAlchemy Documentation. 2024. https://docs.sqlalchemy.org/

4 PostgreSQL Documentation. 2024. https://www.postgresql.org/docs/

5 JWT — JSON Web Tokens. Auth0, 2024. https://jwt.io/introduction

6 Python Software Foundation. Python 3 Documentation, 2024. https://docs.python.org/3/

7 Politecnico di Milano. Software Engineering for Geoinformatics — Project Assignment Specification, Academic Year 2025/26.

8 Group 9. MeteoCity Requirement Analysis and Specification Document (RASD), June 2026. https://github.com/Ayman1017/Software-Engineering-Project-Group_9

9 Group 9. MeteoCity Design Document (DD), June 2026. https://github.com/Ayman1017/Software-Engineering-Project-Group_9

Appendix A — Quick Start Summary

Complete setup and run sequence for MeteoCity on a clean machine:

```
# 1. Clone the repository
git clone https://github.com/Ayman1017/Software-Engineering-Project-Group_9.git
cd Software-Engineering-Project-Group_9

# 2. Install dependencies (Anaconda environment)
pip install -r requirements.txt

# 3. Create the PostgreSQL database
# In psql: CREATE DATABASE meteocity;

# 4. Create backend/.env from the template
copy backend\.env.example backend\.env
# Edit backend/.env: set DATABASE_URL, JWT_SECRET_KEY, RAPIDAPI_KEY

# 5. Terminal 1 -- Start the Flask backend
cd backend
python app.py

# 6. Terminal 2 -- Start the dashboard (from project root)
streamlit run dashboard\meteocity_dashboard.py

# 7. Open browser: http://localhost:8501
```

Appendix B — Environment File Template

The following is the content of backend/.env.example as committed to the repository. Copy this file to backend/.env and replace all placeholder values with real credentials. Never commit backend/.env to version control.

```
# MeteoCity -- Backend Environment Configuration
# Copy this file to backend/.env and fill in your values.
# NEVER commit backend/.env to GitHub -- it is listed in .gitignore.

# PostgreSQL connection string
# Database name must be 'meteocity' (NOT 'meteocity_db')
DATABASE_URL=postgresql://postgres:<your_password>@localhost:5432/meteocity

# Secret key for JWT token signing -- use a long random string
JWT_SECRET_KEY=change-this-to-a-long-random-secret

# RapidAPI key for live Meteostat data ingestion
# Leave empty or keep placeholder to run in mock (synthetic data) mode
RAPIDAPI_KEY=your-rapidapi-key-here
RAPIDAPI_HOST=meteostat.p.rapidapi.com

# Flask application environment
FLASK_ENV=development
```